1 | SQL SELECT Queries Constraints - Activity

Activity:

Activity Number	Session
Activity 01: Select Queries	SQLBolt - Learn SQL - SQL Lesson 1: SELECT queries 101
Activity 02: Queries with Constraints (Pt.1)	SQLBolt - Learn SQL - SQL Lesson 2: Queries with constraints (Pt. 1)
Activity 03: Queries with Constraints (Pt. 2)	SQLBolt - Learn SQL - SQL Lesson 3: Queries with constraints (Pt. 2)

Activity 01: Select Queries

To retrieve data from a SQL database, we need to write **SELECT** statements, which are often colloquially referred to as *queries*. A query in itself is just a statement which declares what data we are looking for, where to find it in the database, and optionally, how to transform it before it is returned. It has a specific syntax though, which is what we are going to learn in the following exercises.

As we mentioned in the introduction, you can think of a table in SQL as a type of an entity (ie. Dogs), and each row in that table as a specific *instance* of that type (ie. A pug, a beagle, a different colored pug, etc).
This means that the columns would then represent the common properties shared by all instances of that entity (ie. Color of fur, length of tail, etc).

And given a table of data, the most basic query we could write would be one that selects for a couple columns (properties) of the table with all the rows (instances).

Select query for a specific columns

```
SELECT column, another_column, …
FROM mytable;
```

The result of this query will be a two-dimensional set of rows and columns, effectively a copy of the table, but only with the columns that we requested.

If we want to retrieve absolutely all the columns of data from a table, we can then use the asterisk (*) shorthand in place of listing all the column names individually.

Select query for all columns

```
SELECT *
FROM mytable;
```

This query, in particular, is really useful because it's a simple way to inspect a table by dumping all the data at once.

EXERCISE

We will be using a database with data about some of Pixar's classic movies for most of our exercises. This first exercise will only involve the Movies table, and the default query below currently shows all the properties of each movie. To continue onto the next lesson, alter the query to find the exact information we need for each task.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 1 — Tasks

1. Find the **title** of each film
2. Find the **director** of each film
3. Find the **title** and **director** of each film
4. Find the **title** and **year** of each film
5. Find **all** the information about each film

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

INFO

Default query -> SELECT * FROM movies;

To find the title of each film

SELECT title FROM movies;

To find the director of each film

SELECT director FROM movies;

To find the title and director of each film

SELECT title, director FROM movies;

To find the title and year of each film

SELECT title, year FROM movies;

To find all the information about each film

SELECT * FROM movies;

Activity 02: Queries with Constraints (Pt. 1)

Now we know how to select for specific columns of data from a table, but if you had a table with a hundred million rows of data, reading through all the rows would be inefficient and perhaps even impossible.

In order to filter certain results from being returned, we need to use a WHERE clause in the query. The clause is applied to each row of data by checking specific column values to determine whether it should be included in the results or not.

Select query with constraints

```
SELECT column, another_column, …
FROM mytable
WHERE condition
    AND/OR another_condition
    AND/OR …;
```

More complex clauses can be constructed by joining numerous AND or OR logical keywords (ie. num_wheels >= 4 AND doors <= 2). And below are some useful operators that you can use for numerical data (ie. integer or floating point):

Operator	Condition	SQL Example
=, !=, <, <=, >, >=	Standard numerical operators	col_name **!=** 4
BETWEEN ... AND ...	Number is within range of two values (inclusive)	col_name **BETWEEN** 1.5 **AND** 10.5
NOT BETWEEN ... AND ...	Number is not within range of two values (inclusive)	col_name **NOT BETWEEN** 1 **AND** 10
IN (...)	Number exists in a list	col_name **IN** (2, 4, 6)
NOT IN (...)	Number does not exist in a list	col_name **NOT IN** (1, 3, 5)

In addition to making the results more manageable to understand, writing clauses to constrain the set of rows returned also allows the query to run faster due to the reduction in unnecessary data being returned.

Did you know?

As you might have noticed by now, SQL doesn't require you to write the keywords all capitalized, but as a convention, it helps people distinguish SQL keywords from column and table names, and makes the query easier to read.

EXERCISE

Using the right constraints, find the information we need from the Movies table for each task below.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 2 — Tasks

1. Find the movie with a row `id` of 6
2. Find the movies released in the `year`s between 2000 and 2010
3. Find the movies **not** released in the `year`s between 2000 and 2010
4. Find the first 5 Pixar movies and their release `year`

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the movie with a row id of 6

SELECT * FROM movies WHERE id = '6'

Find the movies released in the years between 2000 and 2010

SELECT * FROM movies WHERE year BETWEEN 2000 AND 2010

Find the movies not released in the years between 2000 and 2010

SELECT * FROM movies WHERE year NOT BETWEEN 2000 AND 2010

Find the first 5 Pixar movies and their release year

SELECT * FROM movies WHERE id IN (1, 2, 3, 4, 5);

Activity 03: Queries with Constraints (Pt. 2)

When writing WHERE clauses with columns containing text data, SQL supports a number of useful operators to do things like case-insensitive string comparison and wildcard pattern matching. We show a few common text-data specific operators below:

Operator	Condition	Example
=	Case sensitive exact string comparison (**notice the single equals**)	col_name = "abc"
!= or <>	Case sensitive exact string inequality comparison	col_name **!=** "abcd"
LIKE	Case insensitive exact string comparison	col_name **LIKE** "ABC"
NOT LIKE	Case insensitive exact string inequality comparison	col_name **NOT LIKE** "ABCD"
%	Used anywhere in a string to match a sequence of zero or more characters (only with LIKE or NOT LIKE)	col_name **LIKE** "%AT%" (matches "AT", "ATTIC", "CAT" or even "BATS")
_	Used anywhere in a string to match a single character (only with LIKE or NOT LIKE)	col_name **LIKE** "AN_" (matches "AND", but not "AN")
IN (...)	String exists in a list	col_name **IN** ("A", "B", "C")
NOT IN (...)	String does not exist in a list	col_name **NOT IN** ("D", "E", "F")

Did you know?

All strings must be quoted so that the query parser can distinguish words in the string from SQL keywords.

We should note that while most database implementations are quite efficient when using these operators, full-text search is best left to dedicated libraries like Apache Lucene or Sphinx. These libraries are designed specifically to do full text search, and as a result are more efficient and can support a wider variety of search features including internationalization and advanced queries.

EXERCISE

Here's the definition of a query with a WHERE clause again, go ahead and try and write some quotes with the operators above to limit the results to the information we need in the tasks below.

Select query with constraints

```
SELECT column, another_column, …
FROM mytable
WHERE condition
    AND/OR another_condition
    AND/OR …;
```

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 3 — Tasks

1. Find all the Toy Story movies
2. Find all the movies directed by John Lasseter
3. Find all the movies (and director) not directed by John Lasseter
4. Find all the WALL-* movies

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find all the Toy Story movies

SELECT * FROM movies WHERE title LIKE "Toy Story%";

Find all the movies directed by John Lasseter

SELECT * FROM movies WHERE director LIKE "John Lasseter";

Find all the movies (and director) not directed by John Lasseter

SELECT * FROM movies WHERE director NOT LIKE "John Lasseter";

Find all the WALL-* movies

SELECT * FROM movies WHERE title LIKE "WALL-_";

2 | SQL Exploring Joins - Efficient Data Retrieval

Activity:

Activity Number	Session
Activity 04: Filtering and sorting Query results	SQLBolt - Learn SQL - SQL Lesson 4: Filtering and sorting Query results
Activity 05: Simple SELECT Queries	SQLBolt - Learn SQL - SQL Review: Simple SELECT Queries
Activity 06: Multi-table queries with JOINS	SQLBolt - Learn SQL - SQL Lesson 6: Multi-table queries with JOINs
Activity 07: OUTER JOINs	SQLBolt - Learn SQL - SQL Lesson 7: OUTER JOINs

Activity 04: Filtering and sorting Query results

Even though the data in a database may be unique, the results of any particular query may not be - take our Movies table for example, many different movies can be released the same year. In such cases, SQL provides a convenient way to discard rows that have a duplicate column value by using the **DISTINCT** keyword.

Select query with unique results

```
SELECT DISTINCT column, another_column, …
FROM mytable
WHERE condition(s);
```

Since the DISTINCT keyword will blindly remove duplicate rows, we will learn in a future lesson how to discard duplicates based on specific columns grouping and the GROUP BY clause.

ORDERING RESULTS

Unlike our neatly ordered table in the last few lessons, most data in real databases are added in no particular column order. As a result, it can be difficult to read through and understand the results of a query as the size of a table increases to thousands or even million rows.

To help with this, SQL provides a way to sort your results by a given column in ascending or descending order using the **ORDER BY** clause.

Select query with ordered results

```
SELECT column, another_column, …
FROM mytable
WHERE condition(s)
ORDER BY column ASC/DESC;
```

When an **ORDER BY** clause is specified, each row is sorted alpha-numerically based on the specified column's value. In some databases, you can also specify a collation to better sort data containing international text.

LIMITING RESULTS TO A SUBSET

Another clause which is commonly used with the **ORDER BY** clause are the **LIMIT** and **OFFSET** clauses, which are a useful optimization to indicate to the database the subset of the results you care about.

The **LIMIT** will reduce the number of rows to return, and the optional **OFFSET** will specify where to begin counting the number of rows from.

Select query with limited rows

```
SELECT column, another_column, …
FROM mytable
WHERE condition(s)
ORDER BY column ASC/DESC
LIMIT num_limit OFFSET num_offset;
```

If you think about websites like Reddit or Pinterest, the front page is a list of links sorted by popularity and time, and each subsequent page can be represented by sets of links at different offsets in the database.
Using these clauses, the database can then execute queries faster and more efficiently by processing and returning only the requested content.

Did you know?

*If you are curious about when the **LIMIT** and **OFFSET** are applied relative to the other parts of a query, they are generally done last after the other clauses have been applied. We'll touch more on this Lesson 12: Order and execution after introducing a few more parts of the query.*

EXERCISE

There are a few concepts in this lesson, but all are pretty straight-foward to apply. To spice things up, we've gone and scrambled the Movies table for you in the exercise to better mimic what kind of data you might see in real life. Try and use the necessary keywords and clauses introduced above in your queries.

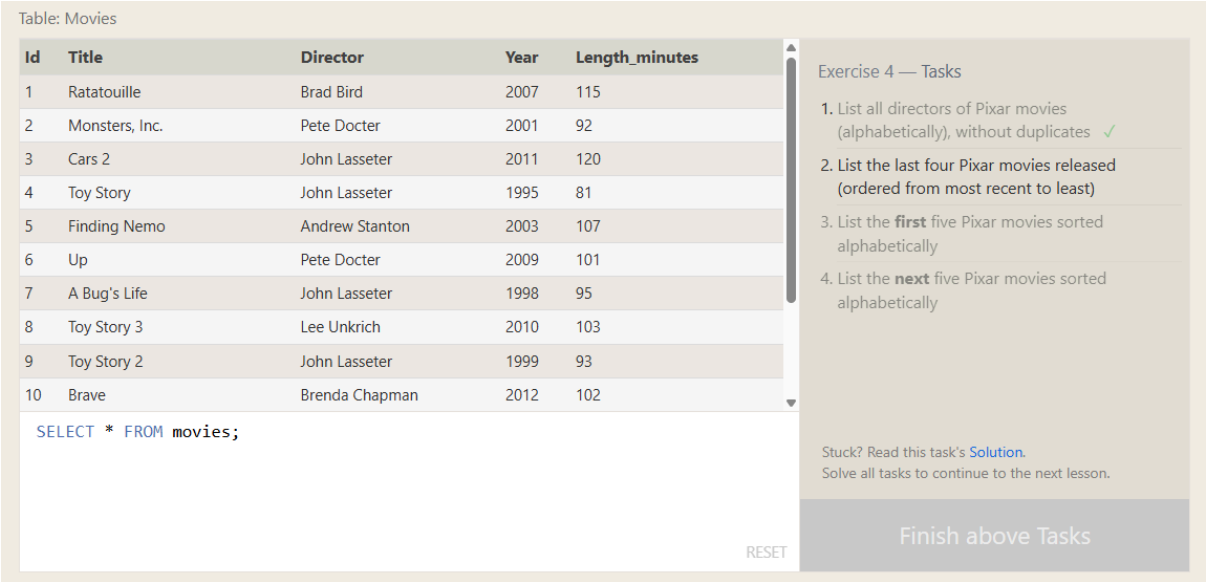


Table: Movies

Id	Title	Director	Year	Length_minutes
1	Ratatouille	Brad Bird	2007	115
2	Monsters, Inc.	Pete Docter	2001	92
3	Cars 2	John Lasseter	2011	120
4	Toy Story	John Lasseter	1995	81
5	Finding Nemo	Andrew Stanton	2003	107
6	Up	Pete Docter	2009	101
7	A Bug's Life	John Lasseter	1998	95
8	Toy Story 3	Lee Unkrich	2010	103
9	Toy Story 2	John Lasseter	1999	93
10	Brave	Brenda Chapman	2012	102

```
SELECT * FROM movies;
```

RESET

Exercise 4 — Tasks

1. List all directors of Pixar movies (alphabetically), without duplicates ✓
2. List the last four Pixar movies released (ordered from most recent to least)
3. List the **first** five Pixar movies sorted alphabetically
4. List the **next** five Pixar movies sorted alphabetically

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

List all directors of Pixar movies (alphabetically), without duplicates

SELECT DISTINCT director FROM movies ORDER BY director ASC;

List the last four Pixar movies released (ordered from most recent to least)

SELECT * FROM movies ORDER BY year DESC LIMIT 4;

List the first five pixar movies sorted alphabetically

SELECT title FROM movies ORDER BY title ASC LIMIT 5;

List the next five Pixar movies sorted alphabetically

SELECT * FROM movies ORDER BY title ASC LIMIT 5 OFFSET 5;

Activity 05: Simple SELECT Queries

You've done a good job getting to this point! Now that you've gotten a taste of how to write basic query, you need to practice writing queries that solve actual problems.

SELECT query

```
SELECT column, another_column, …
FROM mytable
WHERE condition(s)
ORDER BY column ASC/DESC
LIMIT num_limit OFFSET num_offset;
```

EXERCISE

In the exercise below, you will be working with a different table. This table instead contains information about a few of the most populous cities of North America including their population and geo-spatial location in the world.

Did you know?

Positive latitudes correspond to the northern hemisphere, and positive longitudes correspond to the eastern hemisphere. Since North America is north of the equator and west of the prime meridian, all of the cities in the list have positive latitude and negative longitudes.

Try and write some queries to find the information requested in the tasks below. You may have to use a different combination of clauses in your query for each task. Once you're done, continue onto the next lesson to learn about queries that span multiple tables.

Table: North_american_cities

City	Country	Population	Latitude	Longitude
Guadalajara	Mexico	1500800	20.659699	-103.349609
Toronto	Canada	2795060	43.653226	-79.383184
Houston	United States	2195914	29.760427	-95.369803
New York	United States	8405837	40.712784	-74.005941
Philadelphia	United States	1553165	39.952584	-75.165222
Havana	Cuba	2106146	23.05407	-82.345189
Mexico City	Mexico	8555500	19.432608	-99.133208
Phoenix	United States	1513367	33.448377	-112.074037
Los Angeles	United States	3884307	34.052234	-118.243685
Ecatepec de Morelos	Mexico	1742000	19.601841	-99.050674

```
SELECT * FROM north_american_cities;
```

RESET

Review 1 — Tasks

1. List all the Canadian cities and their populations
2. Order all the cities in the United States by their latitude from north to south
3. List all the cities west of Chicago, ordered from west to east
4. List the two largest cities in Mexico (by population)
5. List the third and fourth largest cities (by population) in the United States and their population

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

List all the Canadian cities and their populations

SELECT * FROM north_american_cities WHERE country LIKE “canada”;

Order all the cities in the United States by their latitude from north to south

SELECT * FROM north_american_cities WHERE country LIKE “united states” ORDER BY latitude DESC;

List all the cities west of Chicago, ordered from west to east

SELECT * FROM north_american_cities WHERE longitude <=”-95” ORDER BY longitude ASC;

List the two largest cities in Mexico (by population)

SELECT * FROM north_american_cities WHERE country LIKE “Mexico” ORDER BY population DESC LIMIT 2;

List the third and fourth largest cities (by population) in the United States and their population

SELECT * FROM north_american_cities WHERE country LIKE “united states” ORDER BY population DESC LIMIT 2 OFFSET 2;

Activity 06: Multi-table queries with JOINS

Up to now, we've been working with a single table, but entity data in the real world is often broken down into pieces and stored across multiple orthogonal tables using a process known as *normalization*.

DATABASE NORMALIZATION

Database normalization is useful because it minimizes duplicate data in any single table, and allows for data in the database to grow independently of each other (ie. Types of car engines can grow independent of each type of car). As a trade-off, queries get slightly more complex since they have to be able to find data from different parts of the database, and performance issues can arise when working with many large tables.

In order to answer questions about an entity that has data spanning multiple tables in a normalized database, we need to learn how to write a query that can combine all that data and pull out exactly the information we need.

Multi-table queries with JOINs

Tables that share information about a single entity need to have a *primary key* that identifies that entity *uniquely* across the database. One common primary key type is an auto-incrementing integer (because they are space efficient), but it can also be a string, hashed value, so long as it is unique.

Using the **JOIN** clause in a query, we can combine row data across two separate tables using this unique key. The first of the joins that we will introduce is the **INNER JOIN**.

Select query with INNER JOIN on multiple tables

```
SELECT column, another_table_column, …
FROM mytable
INNER JOIN another_table
    ON mytable.id = another_table.id
WHERE condition(s)
ORDER BY column, … ASC/DESC
LIMIT num_limit OFFSET num_offset;
```

The **INNER JOIN** is a process that matches rows from the first table and the second table which have the same key (as defined by the **ON** constraint) to create a result row with the combined columns from both tables. After the tables are joined, the other clauses we learned previously are then applied.

EXERCISE

We've added a new table to the Pixar database so that you can try practicing some joins. The **BoxOffice** table stores information about the ratings and sales of each particular Pixar movie, and the **Movie_id** column in that table corresponds with the **Id** column in the **Movies** table 1-to-1.
Try and solve the tasks below using the **INNER JOIN** introduced above.

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000
3	7.9	245852179	239163000
6	8	261441092	370001000

Query Results

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 6 — Tasks

1. Find the domestic and international sales for each movie
2. Show the sales numbers for each movie that did better internationally rather than domestically
3. List all the movies by their ratings in descending order

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the domestic and international sales for each movie

SELECT * FROM movies INNER JOIN boxoffice ON id = movie_id;

Show the sales numbers for each movie that did better internationally rather than domestically

SELECT * FROM movies INNER JOIN boxoffice ON id = movie_id WHERE international_sales > domestic_sales;

List all the movies by their ratings in descending order.

SELECT * FROM movies INNER JOIN boxoffice ON id = movie_id ORDER BY rating DESC;

Activity 07: OUTER JOINs

Depending on how you want to analyze the data, the **INNER JOIN** we used last lesson might not be sufficient because the resulting table only contains data that belongs in both of the tables.

If the two tables have asymmetric data, which can easily happen when data is entered in different stages, then we would have to use a **LEFT JOIN**, **RIGHT JOIN** or **FULL JOIN** instead to ensure that the data you need is not left out of the results.

Select query with LEFT/RIGHT/FULL JOINs on multiple tables

```
SELECT column, another_column, …
FROM mytable
INNER/LEFT/RIGHT/FULL JOIN another_table
    ON mytable.id = another_table.matching_id
WHERE condition(s)
ORDER BY column, … ASC/DESC
LIMIT num_limit OFFSET num_offset;
```

Like the **INNER JOIN** these three new joints have to specify which column to join the data on.
When joining table A to table B, a **LEFT JOIN** simply includes rows from A regardless of whether a matching row is found in B.
The **RIGHT JOIN** is the same, but reversed, keeping rows in B regardless of whether a match is found in A.
Finally, a **FULL JOIN** simply means that rows from both tables are kept, regardless of whether a matching row exists in the other table.

When using any of these new joins, you will likely have to write additional logic to deal with **NULL**s in the result and constraints (more on this in the next lesson).

Did you know?

*You might see queries with these joins written as **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, or **FULL OUTER JOIN**, bu the **OUTER** keyword is really kept for SQL-92 compatibility and these queries are simply equivalent to **LEFT JOIN**, **RIGHT JOIN**, and **FULL JOIN** respectively.*

EXERCISE

In this exercise, you are going to be working with a new table which stores fictional data about **Employees** in the film studio and their assigned office **Buildings**. Some of the buildings are new, so they don't have any employees in them yet, but we need to find some information about them regardless.

Since our browser SQL database is somewhat limited, only the **LEFT JOIN** is supported in the exercise below.

Table: Buildings (Read-Only)

Building_name	Capacity
1e	24
1w	32
2e	16
2w	20

Table: Employees (Read-Only)

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2

Query Results

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8
Artist	Jakob J.	2w	6
Artist	Lillia A.	2w	7
Artist	Brandon J.	2w	7

```
SELECT * FROM employees;
```

RESET

Exercise 7 — Tasks

1. Find the list of all buildings that have employees
2. Find the list of all buildings and their capacity
3. List all buildings and the distinct employee roles in each building (including empty buildings)

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the list of all buildings that have employees

SELECT DISTINCT building FROM employees;

Find the list of all buildings and their capacity

SELECT * FROM buildings;

List all buildings and the distinct employee roles in each building (including empty buildings)

SELECT DISTINCT building_name, role FROM building LEFT JOIN employees ON building_name = building;

3 | Nulls & Queries with Expressions

Activity Number	Session
Activity 08: A short note on NULLs	SQLBolt - Learn SQL - SQL Lesson 8: A short note on NULLs
Activity 09: Queries with expressions	SQLBolt - Learn SQL - SQL Lesson 9: Queries with expressions

Activity 08: A short note on NULLs

As promised in the last lesson, we are going to quickly talk about **NULL** values in SQL database.
It's always good to reduce the possibility of **NULL** values in databases because they require special attention when constructing queries, constraints (certain functions behave differently with null values) and when processing the results.

An alternative to **NULL** values in your database is to have **data-type appropriate default values**, like 0 for numerical data, empty strings for text data, etc. But if your database needs to store incomplete data, then **NULL** values can be appropriate if the default values will skew later analysis (for example, when taking averages of numerical data).

Sometimes, it's also not possible to avoid **NULL** values, as we saw in the last lesson when outer-joining two tables with asymmetric data. In these cases, you can test a column for **NULL** values in a **WHERE** clause by using either the **IS NULL** or **IS NOT NULL** constraint.

Select query with constraints on NULL values

```
SELECT column, another_column, …
FROM mytable
WHERE column IS/IS NOT NULL
AND/OR another_condition
AND/OR …;
```

EXERCISE

This exercise will be a sort of review of the last few lessons. We're using the same **Employees** and **Buildings** table from the last lesson, but we've hired a few more people, who haven't yet been assigned a building.

Table: Buildings (Read-Only)

Building_name	Capacity
1e	24
1w	32
2e	16
2w	20

Table: Employees (Read-Only)

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2

Query Results

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8
Artist	Jakob J.	2w	6
Artist	Lillia A.	2w	7
Artist	Brandon J.	2w	7

```
SELECT * FROM employees;
```

RESET

Exercise 8 — Tasks

1. Find the name and role of all employees who have not been assigned to a building
2. Find the names of the buildings that hold no employees

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the name and role of all employees who have not been assigned to a building.

SELECT name, role, building FROM employees LEFT JOIN buildings ON building_na,e = buildings WHERE years_employed = "0";

SELECT name, role FROM employees WHERE years_employed = "0";

Find the names of the buildings that hold no employees.

SELECT * FROM buildings LEFT JOIN employees ON building_name = building WHERE building IS NULL;

Activity 09: Queries with expressions

In addition to querying and referencing raw column data with SQL, you can also use *expressions* to write more complex logic on column values when the query is executed, as shown in this physics example:

Example query with expressions

```
SELECT particle_speed / 2.0 AS half_particle_speed
FROM physics_data
WHERE ABS(particle_position) * 10.0 > 500;
```

Each database has its own supported set of mathematical, string, and date functions that can be used in a query, which you can find in their own respective docs.

The use of expressions can save time and extra post-processing of the result data, but can also make the query harder to read, so we recommend that when expressions are used in the SELECT part of the query, that they are also given a descriptive *alias* using the AS keyword.

Select query with expression aliases

```
SELECT col_expression AS expr_description, …
FROM mytable;
```

In addition to expressions, regular columns and even tables can also have aliases to make them easier to reference in the output and as a part of simplifying more complex queries.

Example query with both column and table name aliases

```
SELECT column AS better_column_name, …
FROM a_long_widgets_table_name AS mywidgets
INNER JOIN widget_sales
  ON mywidgets.id = widget_sales.widget_id;
```

EXERCISE

You are going to have to use expressions to transform the **BoxOffice** data into something easier to understand for the tasks below.

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Table: Boxoffice (Read-Only)

8	8	206445654	417277164
12	6.4	191452396	368400000
3	7.9	245852179	239163000
6	8	261441092	370001000
9	8.5	223808164	297503696
11	8.4	415004880	648167031
1	8.3	191796233	170162503

Query Results

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 9 — Tasks

1. List all movies and their combined sales in **millions** of dollars
2. List all movies and their ratings **in percent**
3. List all movies that were released on even number years

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

List all movies and their combined sales in millions of dollars

SELECT title, (domestic_sales + international_sales) / 1000000 AS gross_sales_millions FROM movies JOIN boxoffice ON movies.id = boxoffice.movie_id;

List all movies and their ratings in percent

SELECT title, rating * 10 AS rating_percent FROM movies JOIN boxoffice ON movies.id = boxoffice.movie_id;

List all movies that were released on even number years

SELECT title, year FROM movies WHERE year % 2 = 0

SELECT * FROM movies WHERE year % 2 = 0

4 | Understanding SQL Queries with Aggregates and Order of Execution

Activity Number	Session
Activity 10: Queries with aggregates (Pt. 1)	SQLBolt - Learn SQL - SQL Lesson 10: Queries with aggregates (Pt. 1)
Activity 11: Queries with aggregates (Pt. 2)	QLBolt - Learn SQL - SQL Lesson 11: Queries with aggregates (Pt. 2)
Activity 12: Order of execution of a query	SQLBolt - Learn SQL - SQL Lesson 12: Order of execution of a Query

Activity 10: Queries with aggregates (Pt. 1)

In addition to the simple expressions that we introduced last lesson, SQL also supports the use of aggregate expressions (or functions) that allow you to summarize information about a group of rows of data. With the Pixar database that you've been using, aggregate functions can be used to answer questions like, "How many movies has Pixar produced?", or "What is the highest grossing Pixar film each year?".

Select query with aggregate functions over all rows

```
SELECT AGG_FUNC(column_or_expression) AS aggregate_description, …
FROM mytable
WHERE constraint_expression;
```

Without a specified grouping, each aggregate function is going to run on the whole set of result rows and return a single value. And like normal expressions, giving your aggregate functions an alias ensures that the results will be easier to read and process.

Common aggregate functions

Here are some common aggregate functions that we are going to use in our examples:

Function	Description
COUNT(*), **COUNT(*column*)**	A common function used to counts the number of rows in the group if no column name is specified. Otherwise, count the number of rows in the group with non-NULL values in the specified column.
MIN(*column*)	Finds the smallest numerical value in the specified column for all rows in the group.
MAX(*column*)	Finds the largest numerical value in the specified column for all rows in the group.
AVG(*column*)	Finds the average numerical value in the specified column for all rows in the group.
SUM(*column*)	Finds the sum of all numerical values in the specified column for the rows in the group.

Grouped aggregate functions

In addition to aggregating across all the rows, you can instead apply the aggregate functions to individual groups of data within that group (ie. box office sales for Comedies vs Action movies).
This would then create as many results as there are unique groups defined as the GROUP BY clause.

Select query with aggregate functions over groups

```
SELECT AGG_FUNC(column_or_expression) AS aggregate_description, …
FROM mytable
WHERE constraint_expression
GROUP BY column;
```

The GROUP BY clause works by grouping rows that have the same value in the column specified.

EXERCISE

For this exercise, we are going to work with our Employees table. Notice how the rows in this table have shared data, which will give us an opportunity to use aggregate functions to summarize some high-level metrics about the teams. Go ahead and give it a shot.

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8
Artist	Jakob J.	2w	6
Artist	Lillia A.	2w	7
Artist	Brandon J.	2w	7

```
SELECT * FROM employees;
```

RESET

Exercise 10 — Tasks

1. Find the longest time that an employee has been at the studio
2. For each role, find the average number of years employed by employees in that role
3. Find the total number of employee years worked in each building

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the longest time that an employee has been at the studio.

SELECT role, name, building, MAX(years_employed) AS years_employed FROM employees;

For each role, find the average number of years employed by employees in that role.

SELECT role, name, AVG(years_employed) AS average_years_employed FROM employees GROUP BY role;

Find the total number of employee years worked in each building.

SELECT building, SUM(years_employed) AS years_employed_per_building FROM employees GROUP BY building;

Activity 11: Queries with aggregates (Pt. 2)

Our queries are getting fairly complex, but we have nearly introduced all the important parts of a SELECT query. One thing that you might have noticed is that if the GROUP BY clause is executed after the WHERE clause (which filters the rows which are to be grouped), then how exactly do we filter the grouped rows?

Luckily, SQL allows us to do this by adding an additional HAVING clause which is used specifically with the GROUP BY clause to allow us to filter grouped rows from the result set.

Select query with HAVING constraint

```
SELECT group_by_column, AGG_FUNC(column_expression) AS aggregate_result_alias, …
FROM mytable
WHERE condition
GROUP BY column
HAVING group_condition;
```

The HAVING clause constraints are written the same way as the WHERE clause constraints, and are applied to the grouped rows. With our examples, this might not seem like a particularly useful construct, but if you imagine data with millions of rows with different properties, being able to apply additional constraints is often necessary to quickly make sense of the data.

Did you know?

If you aren't using the GROUP BY clause, a simple WHERE clause will suffice.

EXERCISE

For this exercise, you are going to dive deeper into Employee data at the film studio. Think about the different clauses you want to apply for each task.

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8
Artist	Jakob J.	2w	6
Artist	Lillia A.	2w	7
Artist	Brandon J.	2w	7

```
SELECT * FROM employees;
```

RESET

Exercise 11 — Tasks

1. Find the number of Artists in the studio (without a **HAVING** clause)
2. Find the number of Employees of each role in the studio
3. Find the total number of years employed by all Engineers

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the number of Artists in the studio (without a HAVING clause).

SELECT role, COUNT(*) as Number_of_artists FROM employees WHERE role = “Artist”;

Find the number of Employees of each role in the studio.

SELECT role, COUNT(role) as Total_employees FROM employees GROUP BY role;

Find the total number of years employed by all Engineers.

SELECT role, SUM(years_employed) AS total_years FROM employees WHERE role = “Engineer”;

Activity 12: Order of Execution of a Query

Now that we have an idea of all the parts of a query, we can now talk about how they all fit together in the context of a complete query.

Complete SELECT query

```
SELECT DISTINCT column, AGG_FUNC(column_or_expression), …
FROM mytable
    JOIN another_table
      ON mytable.column = another_table.column
    WHERE constraint_expression
    GROUP BY column
    HAVING constraint_expression
    ORDER BY column ASC/DESC
    LIMIT count OFFSET COUNT;
```

Each query begins with finding the data that we need in a database, and then filtering that data down into something that can be processed and understood as quickly as possible. Because each part of the query is executed sequentially, it's important to understand the order of execution so that you know what results are accessible where.

<u>Query order of execution</u>

1.- FROM and JOIN s

The FROM clause, and subsequent JOIN s are first executed to determine the total working set of data that is being queried. This includes subqueries in this clause, and can cause temporary tables to be created under the hood containing all the columns and rows of the tables being joined.

2.- WHERE

Once we have the total working set of data, the first-pass WHERE constraints are applied to the individual rows, and rows that do not satisfy the constraint are discarded. Each of the constraints can only access columns directly from the tables requested in the FROM clause. Aliases in the SELECT part of the query are not accessible in most databases since they may include expressions dependent on parts of the query that have not yet executed.

3.- GROUP BY

The remaining rows after the WHERE constraints are applied are then grouped based on common values in the column specified in the GROUP BY clause. As a result of the grouping, there will only be as many rows as there are unique

values in that column. Implicitly, this means that you should only need to use this when you have aggregate functions in your query.

4.- HAVING

If the query has a GROUP BY clause, then the constraints in the HAVING clause are then applied to the grouped rows, discard the grouped rows that don't satisfy the constraint. Like the WHERE clause, aliases are also not accessible from this step in most databases.

5.- SELECT

Any expressions in the SELECT part of the query are finally computed.
6.- DISTINCT

Of the remaining rows, rows with duplicate values in the column marked as DISTINCT will be discarded.

7.- ORDER BY

If an order is specified by the ORDER BY clause, the rows are then sorted by the specified data in either ascending or descending order. Since all the expressions in the SELECT part of the query have been computed, you can reference aliases in this clause.

8.- LIMIT / OFFSET

Finally, the rows that fall outside the range specified by the LIMIT and OFFSET are discarded, leaving the final set of rows to be returned from the query.

Conclusion

Not every query needs to have all the parts we listed above, but part of why SQL is so flexible is that it allows developers and data analysts to quickly manipulate data without having to write additional code, all just by using the above clauses.

EXERCISE

Here ends our lessons on SELECT queries, congrats of making it this far! This exercise will try and test your understanding of queries, so don't be discouraged if you find them challenging. Just try your best.

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000
3	7.9	245852179	239163000
6	8	261441092	370001000

Query Results

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RESET

Exercise 12 — Tasks

1. Find the number of movies each director has directed
2. Find the total domestic and international sales that can be attributed to each director

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Find the number of movies each director has directed

SELECT director, COUNT(id) AS Num_movies_directed FROM movies GROUP BY director;

Find the total domestic and international sales that can be attributed to each director

SELECT director, SUM(domestic_sales + international_sales) AS cumulative_sales_from_all_movies FROM movies INNER JOIN boxoffice ON movies.id = boxoffice.movie_id GROUP BY director;

5 | Managing Data in SQL: Inserting, Updating & Deleting Rows

Activity Number	Session
Activity 13: Inserting rows	SQLBolt - Learn SQL - SQL Lesson 13: Inserting rows
Activity 14: Updating rows	SQLBolt - Learn SQL - SQL Lesson 14: Updating rows
Activity 15: Deleting rows	SQLBolt - Learn SQL - SQL Lesson 15: Deleting rows

Activity 13: Inserting Rows

We've spent quite a few lessons on how to query for data in a database, so it's time to start learning a bit about SQL schemas and how to add new data.

What is a Schema?

We previously described a table in a database as a two-dimensional set of rows and columns, with the columns being the properties and the rows being instances of the entity in the table. In SQL, the *database schema* is what describes the structure of each table, and the datatypes that each column of the table can contain.

> Example: Correlated subquery
>
> For example, in our **Movies** table, the values in the **Year** column must be an Integer, and the values in the **Title** column must be a String.

This fixed structure is what allows a database to be efficient, and consistent despite storing millions or even billions of rows.

Inserting new data

When inserting data into a database, we need to use an INSERT statement, which declares which table to write into, the columns of data that we are filling, and one or more rows of data to insert. In general, each row of data you insert should contain values for every corresponding column in the table. You can insert multiple rows at a time by just listing them sequentially.

Insert statement with values for all columns

```
INSERT INTO mytable
VALUES (value_or_expr, another_value_or_expr, …),
       (value_or_expr_2, another_value_or_expr_2, …),
       …;
```

In some cases, if you have incomplete data and the table contains columns that support default values, you can insert rows with only the columns of data you have by specifying them explicitly.

Insert statement with specific columns

```
INSERT INTO mytable
(column, another_column, …)
VALUES (value_or_expr, another_value_or_expr, …),
      (value_or_expr_2, another_value_or_expr_2, …),
      …;
```

In these cases, the number of values need to match the number of columns specified. Despite this being a more verbose statement to write, inserting values this way has the benefit of being forward compatible. For example, if you add a new column to the table with a default value, no hardcoded INSERT statements will have to change as a result to accommodate that change.

In addition, you can use mathematical and string expressions with the values that you are inserting. This can be useful to ensure that all data inserted is formatted a certain way.

Example Insert statement with expressions

```
INSERT INTO boxoffice
(movie_id, rating, sales_in_millions)
VALUES (1, 9.9, 283742034 / 1000000);
```

Exercise

In this exercise, we are going to play studio executive and add a few movies to the Movies to our portfolio. In this table, the Id is an auto-incrementing integer, so you can try inserting a row with only the other columns defined.

Since the following lessons will modify the database, you'll have to manually run each query once they are ready to go.

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
3	7.9	245852179	239163000
1	8.3	191796233	170162503
2	7.2	162798565	200600000

Query Results

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RUN QUERY RESET

Exercise 13 — Tasks

1. Add the studio's new production, **Toy Story 4** to the list of movies (you can use any director)
2. Toy Story 4 has been released to critical acclaim! It had a rating of **8.7**, and made **340 million domestically** and **270 million internationally**. Add the record to the `BoxOffice` table.

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Add the studio's new production, Toy Story 4 to the list of movies (you can use any director).

INSERT INTO movies VALUES (4, "Toy Story 4", "El Directore", 2015, 90);

INSERT INTO movies (Title, Director, Year, Length_minutes) VALUES ("Toy Story 4", "John Lasseter", "2020", "117");

Toy Story 4 has been released to critical acclaim! It had a rating of 8.7, and made 340 million domestically and 270 million internationally. Add the record to the BoxOffice table.

INSERT INTO boxoffice VALUES (4, 8.7, 340000000, 270000000);

INSERT INTO Boxoffice (Movie_id, rating, domestic_sales, international_sales)
VALUES (“15“, “8.7“, “340000000”, “270000000”);

Activity 14: Updating Rows

In addition to adding new data, a common task is to update existing data, which can be done using an UPDATE statement. Similar to the INSERT statement, you have to specify exactly which table, columns, and rows to update. In addition, the data you are updating has to match the data type of the columns in the table schema.

Update statement with values

```
UPDATE mytable
SET column = value_or_expr,
    other_column = another_value_or_expr,
    …
WHERE condition;
```

The statement works by taking multiple column/value pairs, and applying those changes to each and every row that satisfies the constraint in the WHERE clause.

Taking care

Most people working with SQL will make mistakes updating data at one point or another. Whether it's updating the wrong set of rows in a production database, or accidentally leaving out the WHERE clause (which causes the update to apply to *all* rows), you need to be extra careful when constructing UPDATE statements.

One helpful tip is to always write the constraint first and test it in a SELECT query to make sure you are updating the right rows, and only then writing the column/value pairs to update.

EXERCISE

It looks like some of the information in our Movies database might be incorrect, so go ahead and fix them through the exercises below.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	El Directore	1998	95
3	Toy Story 2	John Lasseter	1899	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RUN QUERY RESET

Exercise 14 — Tasks

1. The director for A Bug's Life is incorrect, it was actually directed by **John Lasseter**
2. The year that Toy Story 2 was released is incorrect, it was actually released in **1999**
3. Both the title and director for Toy Story 8 is incorrect! The title should be "Toy Story 3" and it was directed by **Lee Unkrich**

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

The director for A Bug's Life is incorrect, it was actually directed by John Lasseter

UPDATE movies SET director = "John Lasseter" WHERETitle = "A Bug's Life";

The year that Toy Story 2 was released is incorrect, it was actually released in 1999.

UPDATE movies SET year = "1999" WHERE Title = "Toy Story 2";

Both the title and director for Toy Story 8 is incorrect! The title should be "Toy Story 3" and it was directed by Lee Unkrich

UPDATE movies SET title = "Toy Story 3", director = "Lee Unkrich" WHERE id = "11";

Activity 15: Deleting Rows

When you need to delete data from a table in the database, you can use a DELETE statement, which describes the table to act on, and the rows of the table to delete through the WHERE clause.

Delete statement with condition

```
DELETE FROM mytable
WHERE condition;
```

If you decide to leave out the WHERE constraint, then *all* rows are removed, which is a quick and easy way to clear out a table completely (if intentional).

Taking extra care

Like the UPDATE statement from last lesson, it's recommended that you run the constraint in a SELECT query first to ensure that you are removing the right rows. Without a proper backup or test database, it is downright easy to irrevocably remove data, so always read your DELETE statements twice and execute once.

EXERCISE

The database needs to be cleaned up a little bit, so try and delete a few rows in the tasks below.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RUN QUERY RESET

Exercise 15 — Tasks

1. This database is getting too big, lets remove all movies that were released **before** 2005.
2. Andrew Stanton has also left the studio, so please remove all movies directed by him.

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

This database is getting too big, let's remove all movies that were released before 2005.

DELETE FROM movies WHERE year < 2005;

Andrew Stanton has also left the studio, so please remove all movies directed by him

DELETE FROM movies WHERE director = "Andrew Stanton";

5 | SQL Table Management: Creating, Altering & Dropping Tables

Activity Number	Session
Activity 16: Creating Tables	SQLBolt - Learn SQL - SQL Lesson 16: Creating tables
Activity 17: Altering Tables	SQLBolt - Learn SQL - SQL Lesson 17: Altering tables
Activity 18: Dropping Tables	SQLBolt - Learn SQL - SQL Lesson 18: Dropping tables

Activity 16: Creating Tables

When you have new entities and relationships to store in your database, you can create a new database table using the CREATE TABLE statement.

Create table statement w/ optional table constraint and default value

```
CREATE TABLE IF NOT EXISTS mytable (
    column DataType TableConstraint DEFAULT default_value,
    another_column DataType TableConstraint DEFAULT default_value,
    …
);
```

The structure of the new table is defined by its *table schema*, which defines a series of columns. Each column has a name, the type of data allowed in that column, an *optional* table constraint on values being inserted, and an optional default value.

If there already exists a table with the same name, the SQL implementation will usually throw an error, so to suppress the error and skip creating a table if one exists, you can use the IF NOT EXISTS clause.

Table data types

Different databases support different data types, but the common types support numeric, string, and other miscellaneous things like dates, booleans, or even binary data. Here are some examples that you might use in real code.

Data type	Description
INTEGER , **BOOLEAN**	The integer datatypes can store whole integer values like the count of a number or an age. In some implementations, the boolean value is just represented as an integer value of just 0 or 1.
FLOAT , **DOUBLE** , **REAL**	The floating point datatypes can store more precise numerical data like measurements or fractional values. Different types can be used depending on the floating point precision required for that value.
CHARACTER(num_chars) , **VARCHAR(num_chars)** , **TEXT**	The text based datatypes can store strings and text in all sorts of locales. The distinction between the various types generally amount to underlaying efficiency of the database when working with these columns. Both the CHARACTER and VARCHAR (variable character) types are specified with the max number of characters that they can store (longer values may be truncated), so can be more efficient to store and query with big tables.
DATE , **DATETIME**	SQL can also store date and time stamps to keep track of time series and event data. They can be tricky to work with especially when manipulating data across timezones.
BLOB	Finally, SQL can store binary data in blobs right in the database. These values are often opaque to the database, so you usually have to store them with the right metadata to requery them.

Table constraints

We aren't going to dive too deep into table constraints in this lesson, but each column can have additional table constraints on it which limit what values can be inserted into that column. This is not a comprehensive list, but will show a few common constraints that you might find useful.

Constraint	Description
PRIMARY KEY	This means that the values in this column are unique, and each value can be used to identify a single row in this table.
AUTOINCREMENT	For integer values, this means that the value is automatically filled in and incremented with each row insertion. Not supported in all databases.
UNIQUE	This means that the values in this column have to be unique, so you can't insert another row with the same value in this column as another row in the table. Differs from the `PRIMARY KEY` in that it doesn't have to be a key for a row in the table.
NOT NULL	This means that the inserted value can not be `NULL`.
CHECK (expression)	This allows you to run a more complex expression to test whether the values inserted are valid. For example, you can check that values are positive, or greater than a specific size, or start with a certain prefix, etc.
FOREIGN KEY	This is a consistency check which ensures that each value in this column corresponds to another value in a column in another table. For example, if there are two tables, one listing all Employees by ID, and another listing their payroll information, the `FOREIGN KEY` can ensure that every row in the payroll table corresponds to a valid employee in the master Employee list.

An example

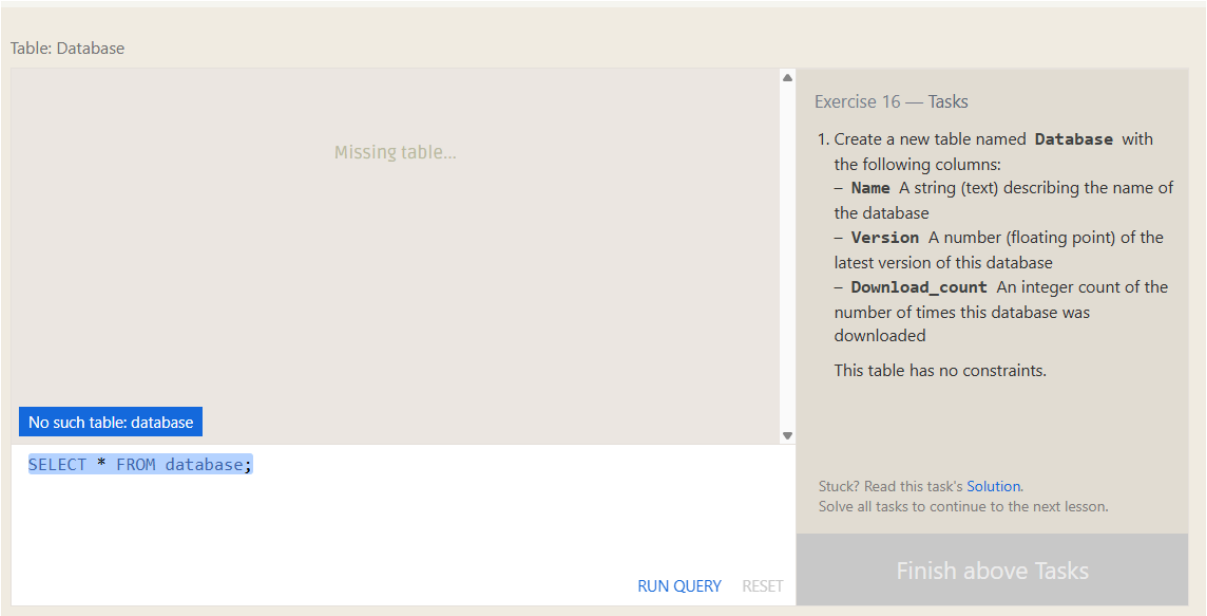
Here's an example schema for the Movies table that we've been using in this lessons up to now.

Movies table schema

```
CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    title TEXT,
    director TEXT,
    year INTEGER,
    length_minutes INTEGER
);
```

EXERCISE

In this exercise, you'll need to create a new table for us to insert some new rols into.

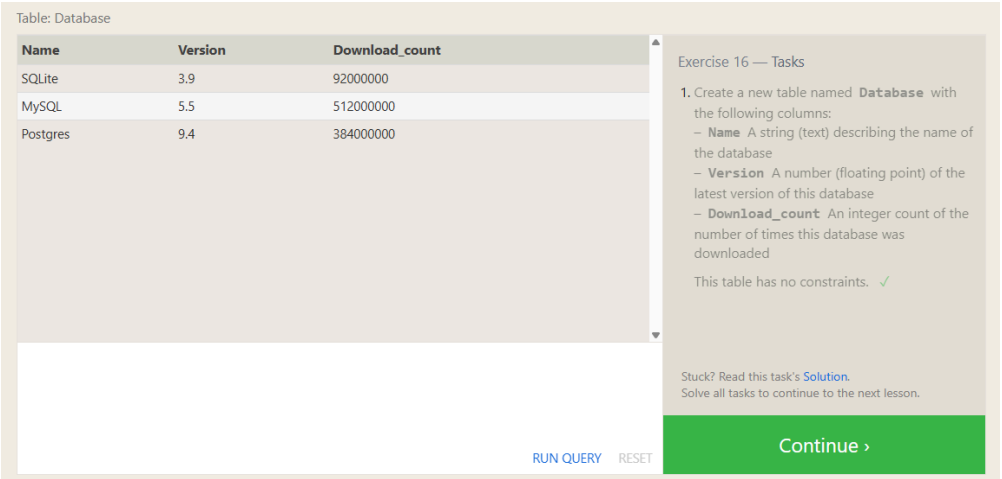


Create a new table named Database with the following columns:

- Name. A string (text) describing the name of the database.
- Version. A number (floating point) of the latest version of this database.
- Download_count. An integer count of the number of times this database was downloaded.

This table has no constraints.

CREATE TABLE Database (Name TEXT, Version FLOAT, Download_count INTEGER);



Activity 17: Altering Tables

As your data changes over time, SQL provides a way for you to update your corresponding tables and database schemas by using the ALTER TABLE statement to add, remove, or modify columns and table constraints.

Adding Columns

The syntax for adding a new column is similar to the syntax when creating new rows in the CREATE TABLE statement. You need to specify the data type of the column along with any potential table constraints and default values to be applied to both existing *and* new rows. In some databases like MySQL, you can even specify where to insert the new column using the FIRST or AFTER clauses, though this is not a standard feature.

Altering table to add new column(s)

```
ALTER TABLE mytable
ADD column DataType OptionalTableConstraint
    DEFAULT default_value;
```

Removing Columns

Dropping columns is as easy as specifying the column to drop, however, some databases (including SQLite) don't support this feature. Instead you may have to create a new table and migrate the data over.

Altering table to remove column(s)

```
ALTER TABLE mytable
DROP column_to_be_deleted;
```

Renaming the table

If you need to rename the table itself, you can also do that using the RENAME TO clause of the statement.

Altering table name

```
ALTER TABLE mytable
RENAME TO new_table_name;
```

Other changes

Each database implementation supports different methods of altering their tables, so it's always best to consult your database docs before proceeding: MySQL, Postgres, SQLite, Microsoft SQL Server.

Exercise

Our exercise use an implementation that only support adding new columns, so give that a try below.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RUN QUERY RESET

Exercise 17 — Tasks

1. Add a column named **Aspect_ratio** with a **FLOAT** data type to store the aspect-ratio each movie was released in.
2. Add another column named **Language** with a **TEXT** data type to store the language that the movie was released in. Ensure that the default for this language is **English**.

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

Add a column named Aspect_ratio with a FLOAT data type to store the aspect-ratio each movie was released in.

ALTER TABLE Movies ADD Aspect_ratio FLOAT;

Add another column named Language with a TEXT data type to store the language that the movie was released in. Ensure that the default for this language is English

ALTER TABLE Movies ADD Language TEXT DEFAULT English;

Activity 18: Dropping Tables

In some rare cases, you may want to remove an entire table including all of its data and metadata, and to do so, you can use the DROP TABLE statement, which differs from the DELETE statement in that it also removes the table schema from the database entirely.

Drop table statement

```
DROP TABLE IF EXISTS mytable;
```

Like the CREATE TABLE statement, the database may throw an error if the specified table does not exist, and to suppress that error, you can use the IF EXISTS clause.

In addition, if you have another table that is dependent on columns in table you are removing (for example, with a FOREIGN KEY dependency) then you will have to either update all dependent tables first to remove the dependent rows or to remove those tables entirely.

Exercise

We've reached the end of our exercises, so let's clean up by removing all the tables we've worked with.

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000
3	7.9	245852179	239163000
6	8	261441092	370001000

Query Results

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

```
SELECT * FROM movies;
```

RUN QUERY RESET

Exercise 18 — Tasks

1. We've sadly reached the end of our lessons, lets clean up by removing the **Movies** table
2. And drop the **BoxOffice** table as well

Stuck? Read this task's Solution.
Solve all tasks to continue to the next lesson.

Finish above Tasks

We've sadly reached the end of our lessons, let's clean up by removing the Movies table

DROP TABLE IF EXISTS Movies;

And drop the BoxOffice table as well

DROP TABLE IF EXISTS Boxoffice;